City Ground Zero

System design: how the city works, why it is built this way, and how its parts coordinate

4	5	15 Hz	1	566
cooperating programs	local message channels	traffic state stream	owner for every fact	sensors and actuators

1. What the system is, and the principles behind it

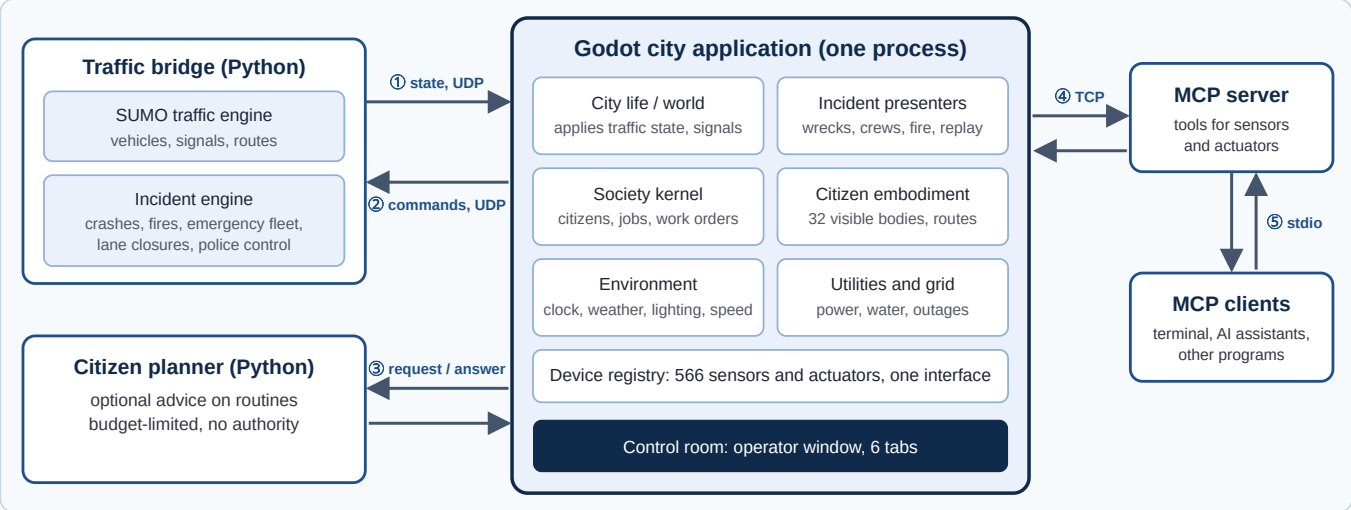
City Ground Zero is a live smart-city simulator: a 3D city with traffic, emergency services, utilities and a society of citizens, instrumented with sensors and actuators that an operator, or any external program, can watch and control. It is not a single simulation but a **federation of specialised simulators and services** that share one world. Five design principles explain almost every choice in the system:

- **Use the right engine for each job.** Road traffic is hard to get right (car following, lane changes, signals, right of way), so it is delegated to SUMO, a validated traffic simulator. The game engine (Godot) draws the world and hosts everything that is spatial or visual. Neither tries to do the other's work.
- **One owner per fact.** Every piece of state (a car's position, a citizen's wage, a reservoir's level) has exactly one authoritative owner. Other parts hold read-only views or ask the owner to change it. This removes contradictions such as a car drawn where the simulator says it is not.
- **Presentation never creates truth.** What you see (a 3D person at a bench) is a rendering of a record that exists in the simulation layer, never the other way round. A citizen does not get a job because a model happens to stand next to a prop.
- **Loose coupling through small messages.** Parts run as separate programs and talk through compact local messages. Any part can be restarted, tested or replaced alone.
- **Determinism and observability.** The population, agendas and work orders are reproducible and every decision carries a stated reason, so behaviour can be explained, replayed and tested.

2. The city plan and zones

The city is an **8 × 8 grid of districts** (64 blocks) with 81 junctions and 144 road links, 49 of them signal-controlled. Blocks are zoned as **downtown** (skyscrapers, hotels, commerce), **residential** (apartments and houses), **commercial and industrial** (workplaces and shops), **parks** (six, with irrigation) and **public services** (seven hospitals, police, emergency stations, six reservoir compounds and two renewable energy hubs). 224 buildings, kerb-aligned street furniture and a ring of overhead power lines complete the plan. The zoning is not decoration: it sets each block's water and power demand, where citizens live and work, and where traffic and incidents concentrate.

3. Components and how they are joined



The four programs are started together by one launcher and talk only through the five channels in the figure; inside Godot the systems are separate nodes that exchange data through Godot's own signals, scene groups and shared object metadata.

4. Movement of data between the parts

#	Channel	Transport	What moves, how often, and why this technique
①	Traffic state: bridge → city	UDP, JSON, one packet per tick (about 15 per second, roughly 40 kB)	Every vehicle (position, heading, speed, size, class, stop reason), every signal head, incident and emergency-unit state, and an operator echo (paused, speed, weather). UDP because only the *latest* state matters: a lost packet is replaced 66 ms later, and there is no queue to back up. Godot decodes the packet on a worker thread and applies only the newest one, so a slow frame never builds a backlog.
②	Commands: city → bridge	UDP, small JSON on a control port	Pause, resume, speed 1–8, weather and its traffic factor, signal phases, and incident requests. Operator commands are idempotent. Incident commands carry a request id; the bridge applies each once and keeps echoing an acknowledgement, while the sender resends every half second until it sees it. This gives *reliable delivery on top of cheap UDP*.
③	Citizen planner	Local HTTP request/answer	A compact summary of a citizen goes out, an advisory suggestion (constrained intent or destination) comes back. It is budget-limited, falls back to a local rule if unavailable, and has no authority: the society kernel decides, so the world never depends on an external service being up.
④	Device interface	TCP, one JSON object per line	Request/answer calls from the MCP server: list devices, read, command. Requests are handled on the main game loop, so a reading is a consistent snapshot of one frame and a command takes effect before the next one.
⑤	MCP	Standard input/output, JSON-RPC 2.0	The industry-standard way for assistants and programs to call tools. Long-running reads send progress notifications and can be cancelled, so a live stream is an ordinary function call.

> **Direction of flow.** Traffic truth flows *out of* SUMO into Godot continuously; intent flows *back* only as small commands. Everything else (citizens, utilities, devices) lives inside Godot and moves by direct reads of the owner's state, which is faster and simpler than messaging within one process.

5. Time and synchronisation

A simulation made of several engines needs one notion of "now". In City Ground Zero **SUMO is the master clock for the physical world**. Each wall-clock tick the bridge runs as many SUMO steps as the chosen speed demands (1–8×), executing *every* step of traffic and incident logic, and publishes only the last, so accelerated runs are never less correct, just coalesced for display. Godot follows: if SUMO cannot deliver the requested speed, the engine's own time scale is eased down to the rate actually delivered, so people, crews, timers and the clock never run ahead of the traffic. Pausing is a command to the bridge, and the pause state is echoed back so every other part freezes together.

On top of physical time sits the **city clock**: a day/night cycle whose pace the operator can set (from real time up to a day in minutes). The environment system owns it and drives lighting and weather; the society kernel advances its own "society minute" from the same clock, and when an operator jumps to a new time of day the change is pushed into the society so agendas, shifts and lighting stay consistent. This separation (physical time for movement, city time for lives) lets the operator fast-forward a day without breaking traffic.

6. Who owns what

Fact	Owner	Everyone else
Position, speed, route of every road vehicle; signal phases; lane closures	SUMO (via the bridge)	Godot renders exactly what is published; vehicle sizes in SUMO equal the 3D models, so a visible contact is a simulated contact
Incident choreography, emergency fleet lifecycle, junction police control	Incident engine (inside the bridge)	Presenters turn the published state into wrecks, crews, fire and replay
Identity, money, jobs, households, agendas, work orders, trips	Society kernel	The embodiment layer shows a body for a citizen and reports completed work; the control room reads
Clock, weather, light level	Environment system	Sensors, traffic factor, lights and the society read it
Power and water stock, demand, outages	Utility system	Meters, lights, HUD and sensors are views of it

7. Traffic and incidents: how an emergency changes the road

Traffic. About 200 vehicles circulate as a closed fleet: they enter at the perimeter and are rerouted legally forever, there is no teleporting and no invisible traffic that rendered cars could queue behind. About 68 of them are drawn in detail at a time. Vehicle classes (car, taxi, bus, police, ambulance, fire, tow, utility) are typed in the packet so each is drawn with the right body. Weather feeds back into behaviour: rain and storm lower a speed factor that SUMO applies to every driver.

Why the incident engine lives next to SUMO. An incident must *physically* change traffic, and only the traffic engine can do that. The engine therefore runs inside the bridge with direct access to SUMO: it choreographs a crash (approach, impact, slide, rest pose), closes lanes, grants controlled overtaking to queued cars, puts nearby junctions under a two-phase police program, reroutes traffic, and drives the emergency fleet. Godot then *presents* what it is told. There are eleven incident types: rear-end, red-light T-bone, unsafe overtake, secondary collision, breakdown, vehicle fire, lost load, building fire, water-main break, signal failure and garbage collection.

Emergency lifecycle. Each unit is always in exactly one state: stationed in its depot bay, dispatched, responding (lights on, permitted to take priority), on scene, then returning. Units are keyed by a stable unit id rather than by their SUMO vehicle, so they can pull out of a bay, drive on the real road network and park again without appearing or vanishing. On scene, presenter crews secure the area, treat people or fight the fire, and tow trucks clear wrecks only after the scene is secured. A recorder captures the published states so an incident can be replayed in a separate render layer that never touches live objects.

8. The society: separating life from appearance

The society kernel simulates **200 citizens in 80 households**, employed by 13 institutions, joined by a graph of about 440 relationships, all created deterministically so a run can be reproduced. Only 32 of them have a 3D body at any moment, but all of them live: their money, fatigue, jobs and agendas change whether or not they are on screen. The separation between the kernel (truth) and the embodiment layer (a body that carries out the truth) is the most important structural decision on the people side.

- **Daily rhythm.** Each citizen has an agenda built from age and employment: sleep, household tasks, commute, work or school, meals, errands and leisure, return home. The activity at any moment is derived from the simulation clock, with the reason recorded. At day close wages and expenses are applied and agendas roll forward.
- **Work orders.** Registered places in the world (a meter cabinet, hydrant, bin, planter) become institutional assets with a condition and a capacity. When an asset needs service the institution opens an order; a compatible, employed, free citizen is assigned with an exclusive reservation of the place. The body walks there over a collision-checked sidewalk graph, performs the work, and reports back; only then does the kernel close the order, change the asset's condition and the institution's budget, pay the citizen and choose the next assignment.
- **Needs and social life.** Thirst, hunger, energy and boredom rise over time and send people to fountains, vending machines, benches or the coffee cart between jobs; afterwards litter goes into the nearest bin that has room. People who meet may stop and talk, and a completed social event strengthens the relationship edge between them.
- **Trips and traffic feedback.** Every movement is a trip record (origin, destination, progress, status). Once a second the kernel stores a snapshot of the live traffic context, labelled clearly as live or as a local fallback, so citizens' explanations can refer to real conditions without pretending to map one person to one SUMO car.
- **Advice without authority.** The optional language-model planner can suggest, but cannot move bodies, change employment or complete an order.

9. Utilities: demand, supply and outages

The utility system models a **42,000 kWh power reserve** and **six reservoirs**. For each of the 64 blocks it computes demand from the actual building mix (a hospital, hotel or skyscraper consumes far more than a house), assigns the block to its nearest reservoir, and groups blocks into **service zones** that share meters and can be switched as a unit. Stocks fall with simulated time at the rate of demand, and a bookkeeping pass about five times a second updates meters, outages and the city HUD. Outages come from two sources: automatic load-shedding when supply is short, and operator cuts of a zone's power or water. Their effects propagate through ordinary data rather than special cases: a cut zone's meters read zero and report "supply offline", its blocks are flagged and highlighted, and lights and sensors that depend on it change on the next sample. The power network is a graph of sources, feeders and three switch cabinets; opening a switch re-evaluates which consumers are still powered.

10. Sensors, actuators and the command path

All 566 devices are registered in one place and read from the live scene on demand: a traffic radar counts the simulated vehicles near it, an air monitor reflects exhaust and rain, a weight sensor on a bin reads what citizens actually threw in. The control room's Sensors tab and every external client therefore see the same numbers. **Commands follow the same path as the operator's buttons**, which is why an external command and a click are indistinguishable: a request is validated against the device's allowed commands, then routed to the owner of that fact.

Actuator	Routed to	Effect
Traffic signals (phase)	Bridge over channel ② → SUMO	All controllers take the phase together; junctions under incident police control keep their safe program
Power switch	Power grid graph	Feeder isolated; downstream consumers re-evaluated and shown unpowered
Service zone cut / restore	Utility system	Zone's power or water supply changes; meters, HUD and outage highlights follow
Street light on / off / auto / dim	Environment lighting pass	Light output follows the override instead of the day/night curve
Weather, clock, speed, pause	Environment system and bridge	Lighting, sensor values and traffic behaviour change; the clock and society are re-aligned
Water and irrigation valves, bin emptying	Device state / waste system	State and indicators change; an emptied bin reads empty at once

Each command answers with the readings before and after, so a caller can verify the effect without a second call.

11. Control room and terminal

The **control room** is a single window containing the live 3D viewport and six tabs: Control (time, weather, speed, camera, signals), Incidents, Assets (highlight any class of object in the city), Utilities, Society and Sensors. It is a *view and a remote control*: it reads public state from the owners and issues the same commands any other client could, which keeps it from becoming a second source of truth. The **MCP terminal** that opens beside it is deliberately just another client of the same interface, with no special access.

12. A scenario end to end

The table follows one scenario, a *rear-end crash during a storm at night*, to show how data moves and which part reacts.

#	Step	What happens
1	Operator sets night and storm	Environment updates the clock and weather, lights and sensors change; a weather command goes to the bridge, which lowers every driver's speed factor; the society is re-aligned to the new time.
2	Incident requested	A tagged command reaches the incident engine over channel ②, is acknowledged, and is retried until acknowledged.
3	Crash choreographed in SUMO	Two vehicles are brought through approach, impact and rest pose; the lane is closed, queued cars are allowed to pass, the nearest junction goes under police control.
4	State published (channel ①)	The next packets carry the wrecks, the closure and the dispatched units; Godot draws the scene and sirens begin.
5	Sensors react	The nearby traffic radar reports occupancy rising and speed falling; air monitors register stopped traffic; the Sensors tab updates on the next sample.
6	Response	Units leave their depots in response mode, secure the scene; crews treat; the tow truck clears the wreck; units return and the closure is lifted.
7	Remote operator	An external client streams the radar, reads the summary of air quality, opens a power switch and watches the affected zone's meters before and after, all through the same interface.

13. Technology choices

Layer	Choice and reason

World, rendering, city systems	**Godot 4**: free, scriptable and fast to iterate; one process hosts world, control room and device interface so internal data moves by direct reads.
Traffic	**Eclipse SUMO** with TraCI: validated car following, lane changing, signals and routing, controllable step by step.
Engines and services	**Python**: the traffic bridge, incident engine, citizen planner and MCP server; quick to write and close to SUMO's tooling.
Messaging	Local **UDP** for streams and commands (latest state wins), **TCP** and **HTTP** for request/answer, **JSON** everywhere for readability; **JSON-RPC over stdio** for MCP.
Assets	Poly Pizza, Poly Haven, Kenney and Quaternius models (CC0 / CC-BY) and Microsoft Rocketbox avatars (MIT), all self-contained in the project.